

LAPORAN PRAKTIKUM KONTROL CERDAS WEEK 5

Nama : Dimas Wahyu Saputra
NIM : 224308080
Kelas : TKA-7D
Akun Github : <https://github.com/DimasWahyuSaputra61003>

1. Judul Percobaan: *Deep Reinforcement Learning untuk Kontrol Kompleks*

2. Tujuan Percobaan:

Tujuan dari percobaan *Deep Reinforcement Learning untuk Kontrol Kompleks* adalah :

- Memahami konsep Deep Reinforcement Learning (DRL) dalam kontrol sistem kompleks.
- Mengimplementasikan Deep Q-Network (DQN) untuk kontrol otomatis.
- Menganalisis performa DRL dibandingkan dengan metode kontrol konvensional.

3. Landasan Teori:

Deep Reinforcement Learning (DRL) merupakan pengembangan dari Reinforcement Learning (RL) yang memanfaatkan neural network untuk memecahkan masalah dengan ruang keadaan (state space) yang kompleks dan kontinyu. Pada dasarnya, RL berfokus pada pembelajaran berbasis pengalaman, di mana agen belajar mengambil keputusan optimal melalui interaksi dengan lingkungan untuk memaksimalkan reward kumulatif. Sementara itu, DRL mengintegrasikan deep learning untuk mempelajari representasi fitur dari data mentah sehingga mampu menangani masalah berskala besar dan dinamis. Menurut (Li, 2018), DRL adalah kombinasi dari deep learning dan reinforcement learning yang memungkinkan agen untuk belajar secara end-to-end dari data sensorik mentah menuju tindakan optimal, tanpa memerlukan model eksplisit dari lingkungan. Pendekatan ini telah berhasil diterapkan pada berbagai bidang seperti pengendalian robot, kendaraan otonom, dan sistem energi pintar (Arulkumaran et al., 2017).

DQN merupakan pengembangan dari algoritma Q-Learning yang menggabungkan Deep Neural Network untuk mengaproksimasi fungsi nilai (Q-value). Q-value merepresentasikan seberapa baik suatu aksi diambil dalam kondisi tertentu. Dengan adanya neural network, DQN dapat menangani state space yang besar dan kompleks yang sulit ditangani Q Learning biasa (Mnih et al., 2015). Dalam sistem kontrol kompleks seperti robotika, kendaraan otonom, dan sistem tenaga listrik, DRL memungkinkan pembelajaran strategi kontrol adaptif tanpa perlu model sistem yang presisi. Dibandingkan dengan metode kontrol konvensional seperti PID atau fuzzy logic, DRL mampu beradaptasi terhadap dinamika lingkungan yang berubah-ubah dan ketidakpastian sistem (Kiumarsi et al., 2014).

4. Diskusi:

1. Apa kelebihan dan kelemahan DQN dibandingkan metode Q-Learning klasik?

Kelebihan DQN: 1. Mengatasi ruang keadaan yang besar

2. Mengekstrak fitur otomatis

3. Generalisasi

4. Kinerja yang unggul pada tugas kompleks

Kelemahan DQN: 1. Ketidakstabilan pelatihan (Training)

2. Keterbatasan pada aksi kontinu

3. Ketergantungan data

2. Bagaimana cara mengadaptasi DQN untuk kontrol sistem dinamis seperti drone atau robot industri?

1) Penyesuaian Ruang Aksi (Action Space)

DQN standar hanya dapat menangani aksi diskret (terhitung). Sistem dinamis seringkali memerlukan aksi kontinu (misalnya, nilai torsi motor, sudut kemudi)

2) Definisi Keadaan (State) dan Imbalan (Reward)

- Keadaan (State):

Input ke DQN harus berupa vektor fitur berdimensi tinggi yang mencakup semua informasi kontrol yang relevan, seperti posisi

(x,y,z), orientasi (roll, pitch, yaw), kecepatan sudut/linear, dan data sensor lainnya.

- Imbalan (Reward): Fungsi imbalan harus dirancang untuk:
Memberikan imbalan positif karena mencapai tujuan (misalnya, mencapai target posisi).

3) Stabilitas Pelatihan

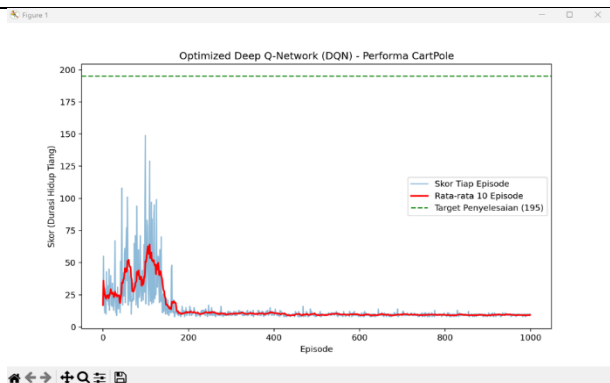
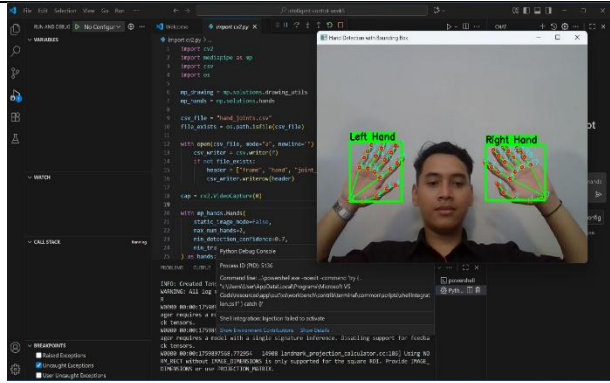
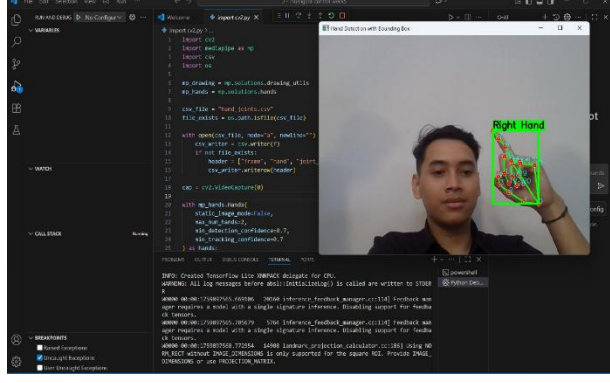
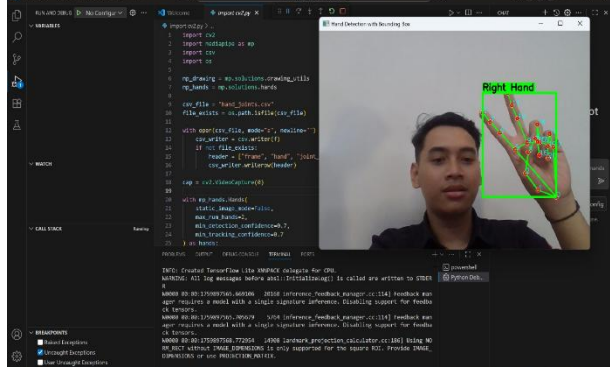
Karena data dari sistem dinamis seringkali bising (noisy)

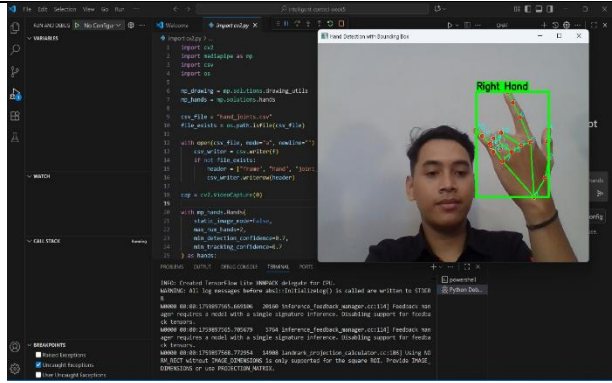
5. *Assignment:*

Pada praktikum kali ini kita membuat sebuah sistem untuk mengumpulkan data gerakan tangan secara langsung dengan memanfaatkan MediaPipe dan OpenCV. Sistem ini berperan dalam mendeteksi keberadaan tangan, menandai lokasi setiap sendi (landmark), dan menyimpan koordinat titik-titik tersebut ke dalam file CSV sebagai kumpulan data. Proses dimulai dengan inisialisasi modul `mp.solutions.hands` dari MediaPipe yang berfungsi untuk mendeteksi tangan, serta `mp.solutions.drawing_utils` yang digunakan untuk menggambar landmark pada video. Sebelum pelacakan dimulai, program mengecek keberadaan file `hand_joints.csv` terlebih dahulu. Apabila file tersebut belum terpenuhi, sistem akan secara otomatis menciptakan file baru dan menyertakan header yang memuat informasi seperti nomor frame, type tangan (kiri atau kanan), ID titik sendi (`joint_id`), serta nilai koordinat x, y, dan z. Kamera diaktifkan melalui `cv2.VideoCapture(0)` untuk menangkap gambar secara langsung. Setiap bingkai yang diambil diubah ke format RGB agar sesuai dengan kebutuhan input.

Prosedur deteksi dilaksanakan dengan model `Hands()`, yang dapat mengidentifikasi hingga dua tangan dengan tingkat keyakinan minimal 0,7 baik untuk pelacakan maupun deteksi. Jika tangan terdeteksi dengan sukses, sistem akan menggambar tanda dan hubungan antar sendi menggunakan fungsi `mp_drawing.draw_landmarks()`. Setiap titik sendi dilabeli dengan nomor (0–20) dan dikelilingi oleh bounding box berwarna hijau agar area tangan yang terdeteksi menjadi lebih jelas. Sistem akan menunjukkan label "Tangan Kanan" atau "Tangan Kiri" berdasarkan hasil klasifikasi dari MediaPipe. Semua data deteksi, termasuk koordinat x, y, dan z dari setiap titik sendi, disimpan secara otomatis ke dalam file CSV pada setiap frame. Dataset ini bisa digunakan pada langkah selanjutnya, contohnya untuk melatih model machine learning seperti SVM dalam mengidentifikasi gerakan tangan. Program beroperasi secara langsung hingga pengguna menekan tombol Esc, setelah itu kamera dan semua jendela aplikasi akan ditutup. Dengan memadukan MediaPipe sebagai alat ekstraksi fitur dan OpenCV untuk visualisasi, sistem ini mampu menciptakan dataset koordinat tangan yang tepat, terorganisir, dan dapat menjadi dasar dalam pengembangan sistem pengenalan gestur tangan menggunakan pembelajaran mesin

6. Data dan Output Hasil Pengamatan:

No	Variabel	Hasil Pengamatan
1	Uji Traning CartPole	
2	Hasil modifikasi dengan 2 tangan	
3	Hasil modifikasi dengan 1 jari	
4	Hasil modifikasi dengan 2 jari	

5	Hasil modifikasi dengan 3 jari	
---	--------------------------------	--

7. Kesimpulan:

Berdasarkan hasil praktikum, sistem pengenalan tangan yang menggunakan MediaPipe dan OpenCV dapat mendeteksi kedua tangan, baik kanan maupun kiri secara langsung dengan tingkat akurasi yang memuaskan. Setiap tangan teridentifikasi melalui 21 titik persendian utama, yang kemudian dihubungkan dengan garis hijau dan ditandai menggunakan titik merah di setiap persendian. Visualisasi deteksi ini tampak kokoh dan seragam, baik ketika hanya satu tangan maupun keduanya muncul dalam satu bingkai gambar. Sistem juga secara otomatis membedakan tangan kiri ("Left Hand") dan tangan kanan ("Right Hand"), serta menyajikan hasil deteksi secara jelas melalui kotak pembatas. Selanjutnya, semua koordinat posisi tangan berhasil dicatat dan disimpan dalam format file CSV, sehingga data tersebut dapat digunakan untuk melatih model klasifikasi gestur, contohnya dengan algoritma Support Vector Machine (SVM). Secara keseluruhan, kombinasi antara MediaPipe dan OpenCV terbukti menjadi solusi yang efisien, cepat, dan terpercaya dalam mendeteksi posisi serta pola gerakan tangan manusia

8. Saran:

- 1) Gunakan algoritma lain seperti PPO (Proximal Policy Optimization) atau DDPG (Deep Deterministic Policy Gradient) untuk mengatasi keterbatasan DQN pada ruang aksi kontinyu.
- 2) Lakukan penyetelan hyperparameter (learning rate, gamma, epsilon, batch size) untuk meningkatkan stabilitas dan konvergensi model.
- 3) Tambahkan perbandingan hasil antar algoritma guna memperkuat analisis performa kontrol.
- d) Uji model pada environment robotik yang lebih kompleks seperti Reacher-v2 atau Manipulator-v1 agar mendekati kondisi nyata.

9. Daftar Pustaka:

- Arulkumaran, K., Deisenroth, M.P., Brundage, M., Bharath, A.A., 2017. A Brief Survey of Deep Reinforcement Learning. IEEE Signal Process. Mag. 34, 26–38. <https://doi.org/10.1109/MSP.2017.2743240>
- Kiumarsi, B., Lewis, F.L., Hamidreza Modares, Karimpour, A., 2014. Reinforcement Q-learning for optimal feedback control. IEEE Trans. Ind. Electron.
- Kumar, A., Levine, A., Feizi, S., 2022. Policy Smoothing for Provably Robust Reinforcement <https://doi.org/10.48550/arXiv.2106.11420>
- Li, Y., 2018. Learning. Deep Reinforcement Learning: An Overview. <https://doi.org/10.48550/arXiv.1701.07274>
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D., 2015. Human level control through deep reinforcement learning. Nature 518, 529–533. <https://doi.org/10.1038/nature14236>